Call Me Maybe

Cours 2 — La tokenisation

---

1. Pourquoi la tokenisation ?

Un LLM ne lit pas des lettres, ni des mots entiers. Il lit des **tokens** — des fragments de mots qui résultent d'un découpage intelligent du texte. C'est crucial pour le projet car le constrained decoding opère **au niveau des tokens**, pas des caractères.

```
Texte brut → [token1, token2, token3, ...] → [ID1, ID2, ID3, ...]
                                → LLM
```

2. Qu'est-ce qu'un token ?

Un token est un fragment de texte auquel correspond un identifiant numérique unique dans un dictionnaire appelé le **vocabulaire**. Ce vocabulaire contient généralement entre 32 000 et 150 000 entrées selon le modèle.

Texte	Tokens produits	Nombre
`fn_add_numbers`	fn | _add | _numbers	3 tokens
`Hello world!`	Hello | _world | !	3 tokens
`anticonstitutionnellement`	ant|icon|stit|ution|nellement	5+ tokens
`{"name": "fn_add"}`	{ | " | name | " | : | ...	7+ tokens

Les mots courants ont souvent leur propre token. Les mots rares sont assemblés depuis des sous-mots.

3. Comment fonctionne le découpage ? (BPE)

L'algorithme le plus courant s'appelle **Byte Pair Encoding (BPE)**. L'idée : on fusionne les paires de caractères les plus fréquentes, encore et encore, jusqu'à avoir un vocabulaire de la taille souhaitée.

Etape	Tokens	Action
Départ	`a a a b d a a a b a c`	Tous les caractères séparés
Étape 1	`[aa] a b d [aa] a b a c`	'aa' est la paire la plus fréquente → fusion
Étape 2	`[aaa] b d [aaa] b a c`	'aa'+'a'='aaa' est fréquent → fusion
Étape 3	`[aaa] [b] d [aaa] [b] a c`	'b' seul reste un token

Résultat : les séquences fréquentes deviennent un seul token ; les séquences rares restent découpées en petits morceaux.

4. Les 3 pièges pour le constrained decoding

Ces détails semblent anodins mais cassent le constrained decoding si on les ignore.

1	**L'espace fait partie du token**	"world" (ID 1917) et " world" (ID 1879) sont deux tokens DIFFERENTS. Dans un JSON, l'espace apres ":" change les tokens valides.
2	**La casse change le token**	"Hello" (ID 9906), "hello" (ID 15339), "HELLO" (ID 13715) = 3 tokens distincts. Chercher "fn_add_numbers" necessite la bonne casse exacte.
3	**Un caractere != un token**	"}" peut etre seul (1 token) ou groupe avec "," en "}," (1 seul token). Il faut verifier si le token correspond au DEBUT de ce qui est attendu.

5. Le vocabulaire JSON dans le projet

Le SDK te donne accès à un fichier JSON via `get_path_to_vocabulary_json()`. C'est un dictionnaire qui mappe chaque token à son ID numérique.

```
# Structure du fichier vocabulaire (simplifie)
{
    "fn":        4083,
    "_add":       914,
    "_numbers":  8271,
    "{":         5265,
    "}":         1602,
    "\"":        334,
    " world":   1879,
    "world":    1917,
    ...
}
```

À chaque étape de génération, tu vas interroger ce vocabulaire pour identifier quels tokens sont autorisés à la position courante dans le JSON.

6. Processus complet — de la phrase au token ID

Etape	Valeur / Appel	Description
Texte brut	`"What is the sum of 2 and 3?"`	Phrase utilisateur
Tokenisation	`["What", " is", " the", " sum", " of", " 2", " and", " 3", "?"]`	encode(texte)
IDs numeriques	`[892, 318, 262, 4771, 286, 16, 290, 17, 30]`	Liste d'entiers
Traitement LLM	`get_logits_from_input_ids(input_ids)`	Tenseur de logits
Logits	`[-2.1, 8.4, 0.3, 5.2, ...]`	Score par token
Selection	`argmax(logits)` → token ID 914 → `'_add'`	Token suivant

Exercice — Reflechis avant de coder

Q1. Le modele doit generer {"name": "fn_. Quels tokens sont valides comme prochaine etape ? Donne des exemples valides ET invalides.

Q2. Pourquoi un token comme "}," (accolade + virgule ensemble) peut poser un probleme dans le constrained decoding, meme si } seul est valide a cette position ?

Q3. Le tokenizer decoupe fn_add_numbers en ["fn", "_add", "_numbers"]. Le modele a deja genere "fn". Quels tokens sont valides pour l'etape suivante ?

Reponses

Q1	Valides : tout token qui peut commencer un nom de fonction — lettres, underscore, "fn", "fn_add", etc. Invalides : }
Q2	Si le tokenizer genere "}," comme un seul token, tu ne peux pas autoriser } sans autoriser aussi },. Il faut travailler a
Q3	Uniquement le token "_add" (et tout token qui commence par _add si le vocabulaire en contient). Le modele doit su

---

A retenir

- Un token = fragment de texte + ID numerique dans un vocabulaire.
- BPE fusionne les paires frequentes : mots courants = 1 token, mots rares = plusieurs tokens.
- L'espace, la casse, et le groupement font partie du token — ils changent l'ID.
- Le vocabulaire JSON du projet est la cle du constrained decoding : il permet de savoir quels tokens sont valides a chaque etape.

Prochaine etape

Cours 3 — Logits et selection de token : comprendre ce que produit le modele a chaque etape, comment transformer des scores bruts en probabilites, et comment on choisit le prochain token (argmax, temperature, sampling).